



FACULTÉ
DES SCIENCES
*Unité de formation
et de recherche*
DÉPARTEMENT
INFORMATIQUE

Université d'Angers

Master 2 Informatique - Conception et Développement

Projet de Fin d'Année

Conception d'un site web pour la gestion des personnels et matériels

Présenté par :

Firmin CAMUS, Antonin CHERRE,
Hugo MAGRET, Clément OUSSET

Encadrants :

M. Jean-Michel RICHER
M. Vincent BARICHARD

Année universitaire 2025 - 2026

Table des matières

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	Enjeux et Périmètre	2
2	Architecture et Choix Techniques	3
2.1	Frontend : Angular	3
2.2	Backend : Node.js	3
2.3	Base de données : PostgreSQL	3
2.3.1	Schéma de fonctionnement	4
2.4	Déploiement et Conteneurisation : Docker	4
2.5	Collaboration et Intégration Continue : GitHub	4
2.6	Organisation du Projet	4
3	Conception de la Base de Données	6
3.1	Modèle Logique de Données (MLD)	6
3.2	Détail des tables et contraintes	7
4	Conception Logicielle	8
4.1	Diagramme de Classes UML	8
4.2	Paradigme de développement	8

Chapitre 1

Introduction

Dans le cadre de notre dernière année de Master 2 en Informatique, parcours Conception et Développement, il nous a été confié la réalisation d'un projet de fin d'études axé sur la gestion du personnel et du matériel du département informatique[cite : 1, 2].

1.1 Contexte et Objectifs

L'objectif principal de ce projet est de concevoir et développer un site web complet permettant de recenser et de gérer efficacement les personnels ainsi que le matériel informatique au sein des différents locaux du département[cite : 6].

Ce système doit permettre de classer les espaces selon leur type (bureau, salle de classe, salle de réunion...) et de gérer finement leur capacité d'accueil[cite : 7, 8]. Il est également crucial de pouvoir suivre l'équipement présent dans chaque salle (prises réseaux, vidéo-projecteurs tableaux...), et de gérer dynamiquement les affectations du personnel[cite : 7, 10].

1.2 Enjeux et Périmètre

Au-delà du simple développement d'un système d'information de type CRUD (Create, Read, Update, Delete), les attentes de ce projet portent sur la maintenabilité de l'application et son ergonomie[cite : 12, 14]. Parmi les fonctionnalités majeures attendues, nous devons implémenter :

- **Une cartographie interactive** : Le système doit permettre de visualiser les locaux via un plan interactif. Bien que des plans au format PDF nous aient été fournis, nous exploiterons principalement leur version vectorielle (SVG), qui sont techniquement beaucoup plus sympathiques à manipuler dans un contexte web. Ils nous permettent en effet de définir facilement des zones cliquables (via des balises HTML/SVG) afin de rendre l'expérience utilisateur plus intuitive.
- Un moteur de génération de plans au format PDF pour exporter l'état d'un étage d'un bâtiment, avec le matériel et les personnels affectés[cite : 16].

Ce rapport détaillera l'ensemble du processus de création de cette plateforme, depuis nos choix technologiques orientés "industrie" jusqu'à la mise en place de notre architecture logicielle.

Chapitre 2

Architecture et Choix Techniques

Pour répondre aux exigences de maintenabilité et de simplicité de déploiement imposées par le sujet [cite : 12, 14], nous avons fait le choix de nous écarter des solutions CMS existantes pour développer une application sur mesure [cite : 11]. Nos choix technologiques ont été guidés par deux critères majeurs : la robustesse de la solution et notre volonté de consolider des compétences demandées dans le monde de l'entreprise.

2.1 Frontend : Angular

Pour l'interface utilisateur, notre choix s'est porté sur le framework Angular. Ce choix s'explique tout d'abord par une compétence déjà présente au sein de l'équipe (deux membres avaient déjà utilisé cette technologie). De plus, Angular est aujourd'hui un standard de l'industrie pour les applications web complexes (Single Page Applications). Sa structure basée sur les composants et l'utilisation de TypeScript garantissent un code typé, maintenable et évolutif.

2.2 Backend : Node.js

Bien que notre formation de 5 ans en informatique nous ait permis d'acquérir une grande expertise sur le langage comme Java, nous avons décidé d'implémenter notre API Backend en Node.js. Ce choix est doublement stratégique. D'une part, Node.js est extrêmement populaire dans l'industrie actuelle pour le développement de micro-services et d'API REST. D'autre part, cela nous permettait de sortir de notre zone de confort académique (Java) pour relever un défi technique pertinent pour notre future insertion professionnelle.

2.3 Base de données : PostgreSQL

Le système devant gérer de potentielles écritures simultanées (modification de salles, ajout de matériels), nous avons rapidement écarté SQLite. Très léger, SQLite nous semblait trop faible pour nos besoins. Notre choix final s'est donc porté sur PostgreSQL, un Système de Gestion de Base de Données Relationnelle (SGBDR) robuste et open-source. Il s'intègre également de manière fluide avec Node.js.

2.3.1 Schéma de fonctionnement

Le schéma suivant illustre le workflow principal de l'application : l'utilisateur interagit avec le frontend Angular, qui transmet des requêtes HTTP au backend Node.js. Ce backend traite la logique métier, interroge PostgreSQL pour lire ou modifier les données, puis renvoie les réponses au frontend.

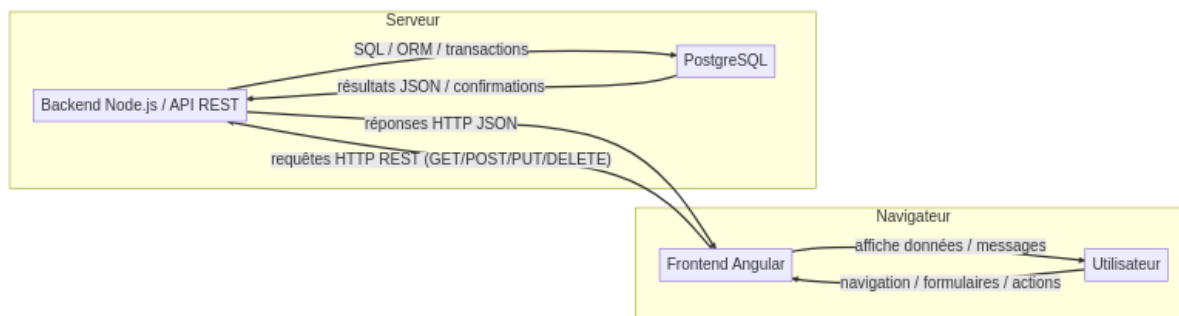


FIGURE 2.1 – Workflow de communication entre l'utilisateur, le frontend, le backend et la base de données

2.4 Déploiement et Conteneurisation : Docker

Afin de garantir que notre site soit "facilement installable" sur n'importe quel serveur du département informatique[cite : 14], nous avons intégré Docker dès le premier jour de développement. Docker nous permet d'encapsuler notre base de données, notre backend et notre frontend dans des conteneurs isolés. L'utilisation de `docker-compose` permet de lancer l'intégralité de l'environnement avec une seule commande, résolvant ainsi les problèmes de compatibilité entre les systèmes d'exploitation, que ce soit pour nous développeurs du projet, ou pour les futurs utilisateurs.

2.5 Collaboration et Intégration Continue : GitHub

Travaillant en équipe de quatre, la gestion des versions du code était critique. Nous avons utilisé GitHub pour coordonner nos développements. EN plus la fusion des fonctionnalités gérée de manière fluide, GitHub nous offre le possibilité de créer des *releases* pour le rendu final, et de créer des tickets (issues) pour répartir les tâches, de manière native.

2.6 Organisation du Projet

Afin de garantir une séparation stricte des responsabilités (separation of concerns), notre dépôt de code est structuré de la manière suivante :

```
Gestion_locaux/
|-- back/           # API Node.js (Logique metier et acces DB)
|-- front/          # Application Angular (Interface utilisateur)
|-- database/       # Fichiers SQL et scripts d'initialisation PostgreSQL
```

```
|-- rapport/          # Sources LaTeX de ce document  
+-- docker-compose.yml # Orchestrateur des differents conteneurs
```

Cette arborescence permet à chaque membre de l'équipe de se repérer facilement et facilite le déploiement automatisé.

Chapitre 3

Conception de la Base de Données

La conception de la base de données est au cœur de notre application. Pour garantir l'intégrité des données des locaux, du personnel et du matériel, nous avons choisi une modélisation relationnelle sous PostgreSQL.

3.1 Modèle Logique de Données (MLD)

Le schéma ci-dessous présente l'organisation des tables et les relations entre elles. Il met l'accent sur les contraintes d'intégrité référentielle (clés primaires et étrangères), qui assurent la cohérence des informations de l'inventaire et de l'occupation des salles.

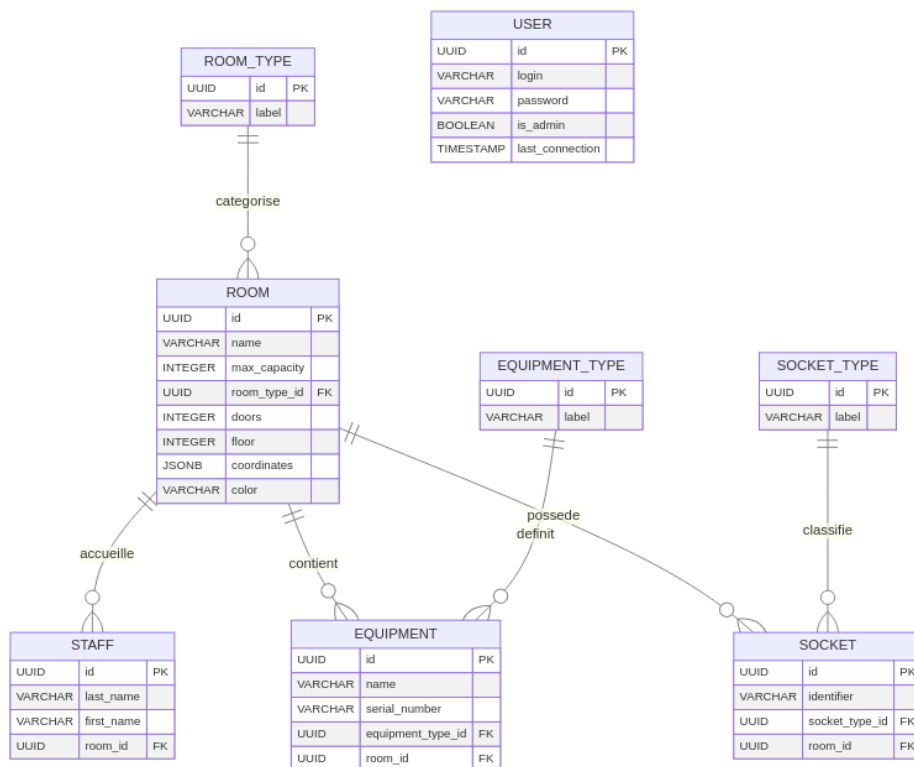


FIGURE 3.1 – Schéma relationnel de la base de données PostgreSQL

3.2 Détail des tables et contraintes

- **Table room** : Elle modélise les espaces physiques. Chaque salle est caractérisée par sa capacité maximale, son type, son étage et ses coordonnées pour le plan.
- **Table staff** : Elle contient le personnel affecté aux locaux. La relation `room_id` permet de savoir où chaque employé est positionné.
- **Table user** : Cette table stocke les comptes d'accès à l'application. Le champ `is_admin` distingue les utilisateurs avec droits d'administration des utilisateurs standards.
- **Tables de référence** : `room_type`, `equipment_type` et `socket_type` garantissent la cohérence des catégories utilisées dans les salles, le matériel et les prises.
- **Table equipment** : Elle représente l'inventaire matériel d'une salle. Chaque équipement est lié à un type et peut être rattaché à une salle.
- **Table socket** : Elle répertorie les prises présentes dans chaque salle et les associe à un type de connectique.

La modélisation adopte des règles solides : les types de référence sont supprimés en cascade, tandis que les dépendances sur les salles sont gérées avec `SET NULL` lorsque cela est pertinent, afin de préserver les données historiques et limiter les pertes involontaires.

Chapitre 4

Conception Logicielle

Après avoir défini la persistance, nous avons structuré l'application en deux couches principales : le backend Node.js, qui fournit l'API REST, et le frontend Angular, qui orchestre l'interface et la manipulation des données côté client.

4.1 Diagramme de Classes UML

Le diagramme ci-dessous illustre la structure objet de l'application Angular et les principaux services métiers. Il montre comment les composants, les modèles et les services collaborent pour gérer les locaux, le personnel, le matériel et les prises.

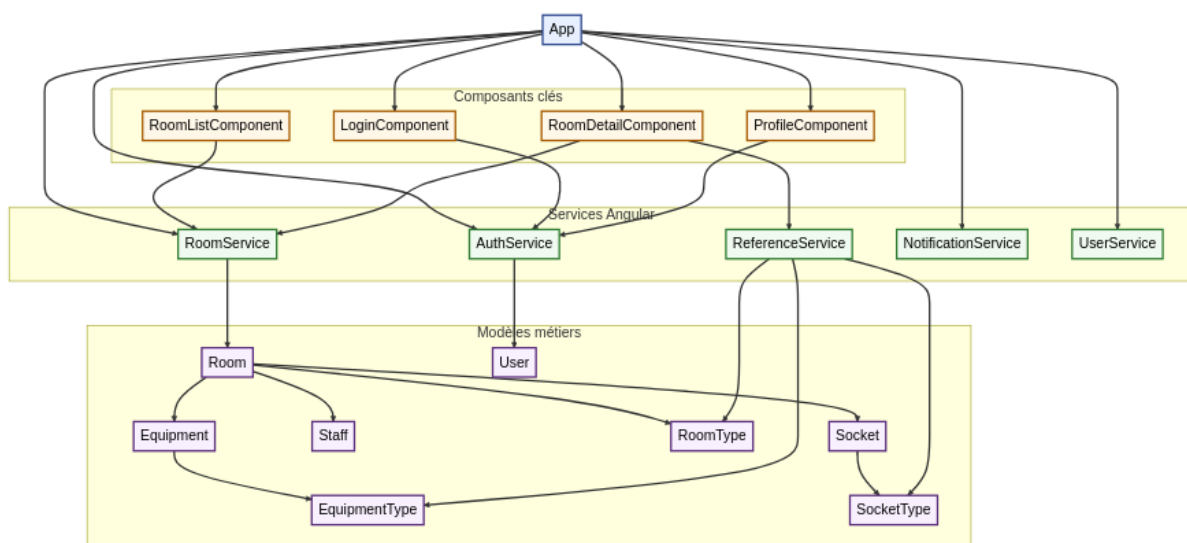


FIGURE 4.1 – Diagramme de classes UML - Architecture applicative Angular

4.2 Paradigme de développement

La conception logicielle privilégie une séparation claire des responsabilités entre :

- **Modèles** : Les classes `Room`, `Equipment`, `Staff`, `RoomType`, `EquipmentType` et `SocketType` définissent les entités manipulées par le client.

- **Services** : Les services Angular (`RoomService`, `AuthService`, `ReferenceService`, `EquipmentService`, `StaffService`, `SocketService`) encapsulent l'accès aux API et la logique de synchronisation des données.
- **Composants** : Les composants gèrent l'affichage et l'interaction utilisateur : liste des salles, détail d'une salle, gestion des types, connexion, etc.

Cette architecture améliore la maintenabilité du projet. Le backend reste léger et modulaire, tandis qu'Angular assure un rendu dynamique et une expérience utilisateur fluide.